



南開大學
Nankai University

南 開 大 學

計 算 機 學 院

並行程序設計實驗報告

CPU 架構編程

錢展飛

年級：2023 級

專業：計算機科學與技術

指導教師：王剛

2026 年 3 月 20 日

目录

一、 $n \times n$ 矩阵与向量内积	1
(一) 实验环境	1
(二) 实验介绍	1
(三) 算法设计与分析	1
1. 逐行访问元素的平凡算法	1
2. cache 优化算法	1
3. 结果统计与分析	2
4. 实验心得	3
二、 n 个数之和	3
(一) 实验介绍	3
(二) 算法设计	3
1. 平凡算法设计及编程实现	3
2. 优化算法	3
(三) 性能测试	4
(四) 结果分析	5
三、 总结与反思	5

一、 n*n 矩阵与向量内积

(一) 实验环境

处理器	13th Gen Intel(R) Core(TM) i5-13420H 2.10 GHz
机带 RAM	16.0 GB (15.6 GB 可用)
设备 ID	93DE6456-BDA4-4AEE-83AA-7365C0D775D2
产品 ID	00342-30958-46058-AAOEM
系统类型	64 位操作系统, 基于 x64 的处理器

(二) 实验介绍

给定一个 $n \times n$ 矩阵, 计算每一列与给定向量的内积, 考虑两种算法设计思路: 逐列访问元素的平凡算法和 cache 优化算法, 进行实验对比:

1. 对两种思路的算法编程实现.
2. 练习使用高精度计时测试程序执行时间, 比较平凡算法和优化算法的性能.

(三) 算法设计与分析

1. 逐行访问元素的平凡算法

矩阵的每一列依次与向量相乘, 逐列访问矩阵元素: 一步外层循环 (内存循环一次完整执行) 计算出一个内积结果。由于当 n 较小时, 程序运行速度过快, 导致时间测量不精准。所以将代码关键部分循环 $times$ 次最后算出平均时间, 以减小误差。

代码如下:

平凡算法

```
1 for (int i = 0; i < n; i++) {  
2     sum[i] = 0;  
3     for (int j = 0; j < n; j++)  
4         sum[i] += b[j][i] * a[j];  
5 }
```

2. cache 优化算法

改为逐行访问矩阵元素: 一步外层循环计算不出任何一个内积, 只是向每个内积累加一个乘法结果。

```
1 for (int i = 0; i < n; i++)  
2     sum[i] = 0;  
3     for (int j = 0; j < n; j++) {
```

```
4     for (int i = 0; i < n; i++) {  
5         sum[i] += b[j][i] * a[j];  
6     }  
7 }  
8 }
```

3. 结果统计与分析

算法执行时间表

矩阵阶数	平凡算法	优化算法
500.0	1.5637	1.03007
1000.0	7.62169	3.27123
1500.0	13.9495	7.21024
2000.0	30.2541	17.4222
2500.0	40.3656	19.9045
3000.0	63.4001	28.0145
3500.0	94.7451	39.2541
4000.0	118.961	56.1254

图 1

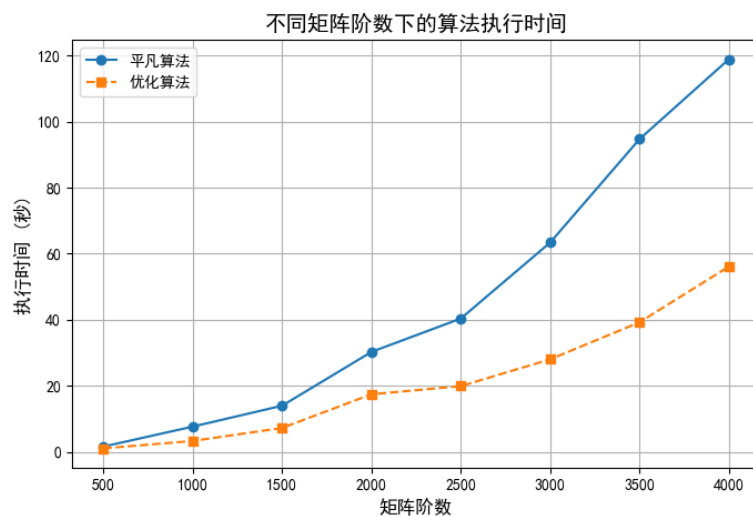


图 2: 算法性能对比

4. 实验心得

显然，优化算法（row-column）算法均比平凡算法（column-row）性能更优。因为优化算法比起平凡算法的访存模式具有更好的空间局部性，使得 cache 的能力得以发挥。由于数组数据在计算机内部是按照行主序存储的，故以 column-row 形式访问地址时会大量出现 Cache 丢失的情况，而 row-column 访问形式由于是顺着存储顺序访问，Cache 丢失情况较少，命中率更高。按行访问，则实际上是连续读取，刚好多次都能够 cache 命中；而按列读取，则是非连续读取，cache 命中的理论可能性不大，所以需要访问主存的次数增多，于是，所花费时间变多。

二、 n 个数之和

（一） 实验介绍

计算 n 个数的和，考虑两种算法设计思路：逐个累加的平凡算法（链式）；适合超标量架构的指令级并行算法（相邻指令无依赖），如最简单的两路链式累加，再如递归算法——两两相加、中间结果再两两相加，依次类推，直至只剩下最终结果。完成如下作业：

1. 对两种算法思路编程实现；
2. 练习使用高精度计时测试程序执行时间，比较平凡算法和优化算法的性能。

（二） 算法设计

1. 平凡算法设计及编程实现

平凡算法

```
1 int sum = 0;
2     for (int i = 0; i < n; i++) {
3         sum += arr[i];
4     }
5     return sum;
```

2. 优化算法

多链路式

```
1     int sum1 = 0;
2     int sum2 = 0;
3     int sum = 0;
4     for (int i = 0; i < n; i += 2) {
5         sum1 += a[i];
6         sum2 += a[i + 1];
7     }
8     sum = sum1 + sum2;
9     return sum;
```

递归优化算法

```

1  if (n == 1) return;
2  else {
3      for (int i = 0; i < (n / 2); i++)
4      {
5          b[i] += b[n - i - 1];
6      }
7      n = (n + 1) / 2;
8      recursion(n, b);
9  }

```

二重循环算法

```

1  int m;
2  for (int m = n; m > 1; ++m /= 2) //++m 避免m为奇数时导致最末端一个数丢失
3  {
4      for (int i = 0; i < m / 2; i++)
5          a[i] = a[i + 2] + a[i * 2 + 1];
6      if (m % 4 == 1) a[m / 4] = a[m - 1];
7  }

```

(三) 性能测试

程序运行时间通过利用 QueryPerformanceFrequency 与 QueryPerformanceCounter 实现毫秒级延时。每种数据规模都进行 40 次测试取平均值作为程序运行时间。

算法执行时间对比（单位：秒）

阶数	平凡算法	二重循环	多路链路	递归
100.0	0.0001475	0.0012225	0.0002575	0.0003175
200.0	0.000265	0.0006875	0.0003375	0.00075
400.0	0.0004875	0.00128	0.0010225	0.001475
800.0	0.00094	0.0024925	0.001155	0.0026525
1600.0	0.001865	0.0047525	0.002445	0.00371
3200.0	0.00369	0.0103125	0.0050475	0.0090525
6400.0	0.018365	0.014635	0.0106475	0.01748
12800.0	0.0340225	0.0291675	0.022165	0.04225
25600.0	0.074335	0.0729875	0.0457475	0.09288
51200.0	0.0603375	0.104667	0.084455	0.144083
102400.0	0.137385	0.249918	0.179783	0.326242
204800.0	0.23646	0.504905	0.341188	0.883788
409600.0	0.721317	0.891858	0.540527	1.62098
819200.0	1.98233	1.70942	1.05476	3.00336
1638400.0	3.42828	3.54179	2.95417	5.9112
3276800.0	6.38098	7.0633	6.40343	7.87793
6553600.0	8.6701	14.1655	6.76137	15.7811

图 3

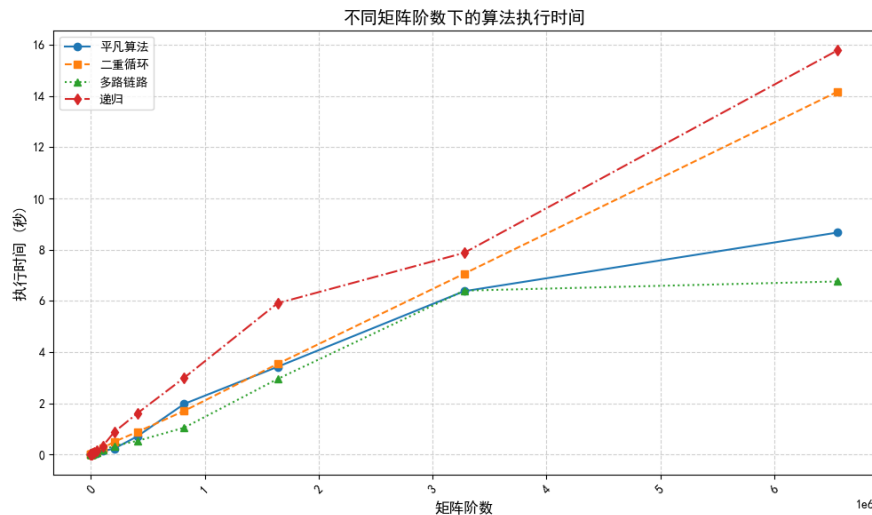


图 4

(四) 结果分析

根据图可以看出，在 Linux-X86 平台上的算法性能整体比 Windows-X86 平台上更优。而在各自平台上，则基本呈现多链路式优于递归，递归略优于双重循环，双重循环优于平凡算法的情况。分析原因：递归的算法虽然简洁，但是是牺牲时间和空间的，因此性能来说，递归不如循环性能好。因为每一次函数调用，都需要在内存中分配空间以保存参数、返回地址及临时变量，而且往栈里面压入数据和弹出数据都需要时间。循环的执行原理是通过改变内部变量和指令跳转实现，所以在该程序中二重循环代码量较大，CPU 刷新次数多。所以递归和二重循环性能相较于多链路式较差。

三、 总结与反思

本次实验围绕矩阵与向量内积计算和 n 个数求和两个核心问题展开，通过对比不同算法的性能表现，深入探讨了计算机体系结构中缓存优化和指令级并行的重要性。实验结果表明通过调整矩阵元素的访问顺序 (row-column vs. column-row)，显著提升了缓存命中率。多链路式累加和递归算法相比平凡累加算法，在超标量架构下充分利用了指令级并行性，性能提升约。其中，多链路式因减少数据依赖，表现最优。同时，测试数据规模有限，未覆盖极端场景（如超大规模矩阵或非对齐内存访问）。

github: <https://github.com/qian-zf/bxlab>